

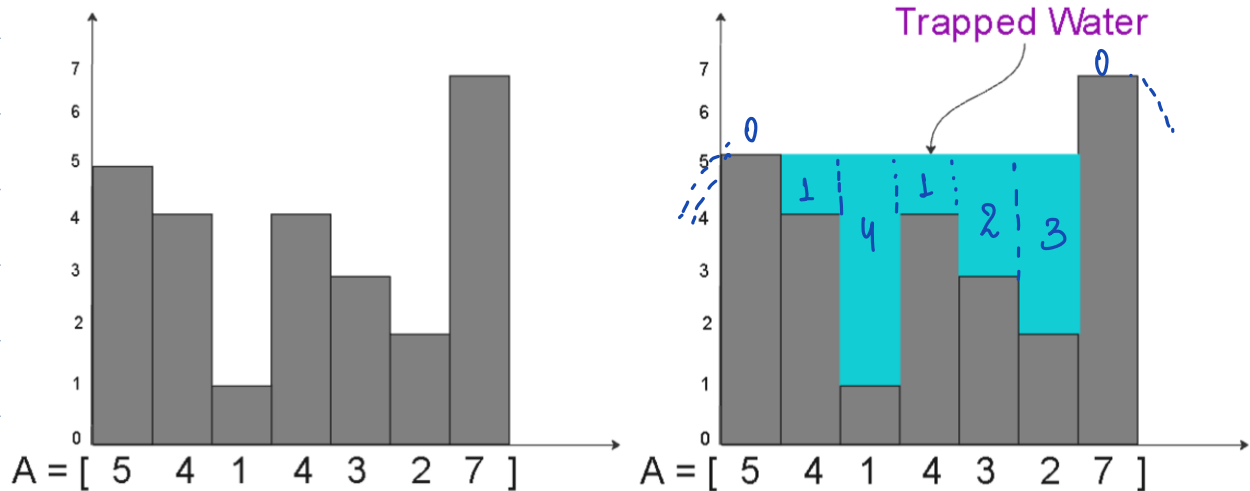
Rain Water Trapped

Imagine a histogram where the bars' heights are given by the array A . Each bar is of uniform width, which is 1 unit. When it rains, water will accumulate in the valleys between the bars.

Your task is to calculate the total amount of water that can be trapped in these valleys.

Example:

The Array $A = [5, 4, 1, 4, 3, 2, 7]$ is visualized as below. The total amount of rain water trapped in A is 11.



Problem Constraints

$$1 \leq |A| \leq 10^5$$

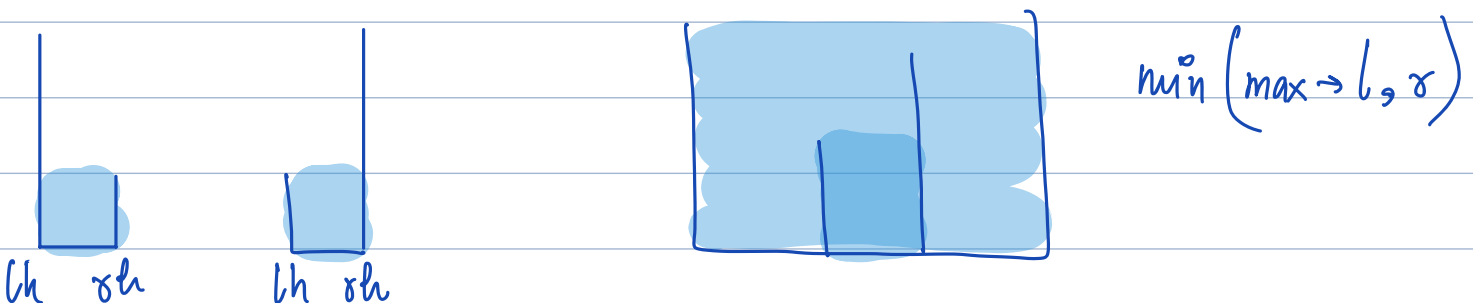
$$0 \leq A[i] \leq 10^5$$

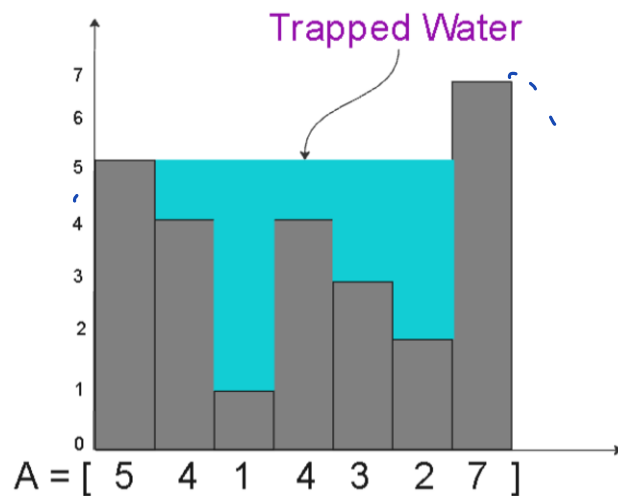
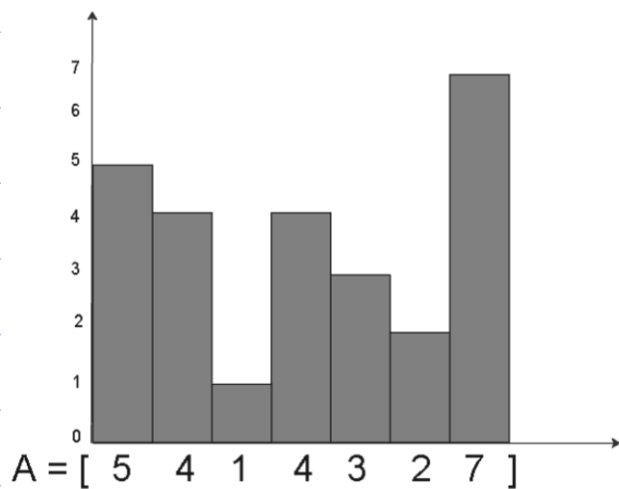
Input Format

First and only argument is the Integer Array, A .

Output Format

Return an Integer, denoting the total amount of water that can be trapped in these valleys





Steps:

1. Find leftmax & rightmax
2. Find $\min(\text{leftmax}, \text{rightmax})$
3. Water += $\min - \text{ht}$

Brute Force

function trapWater(height):

n = length of height

totalWater = 0

// Traverse each bar

for i from 0 to n-1:

leftMax = 0

rightMax = 0

// Find the maximum height on the left of index i

for j from 0 to i:

leftMax = max(leftMax, height[j])

// Find the maximum height on the right of index i

for j from i to n-1:

rightMax = max(rightMax, height[j])

// Water trapped at index i depends on the shorter boundary

waterAti = min(leftMax, rightMax) - height[i]

// Add to total trapped water

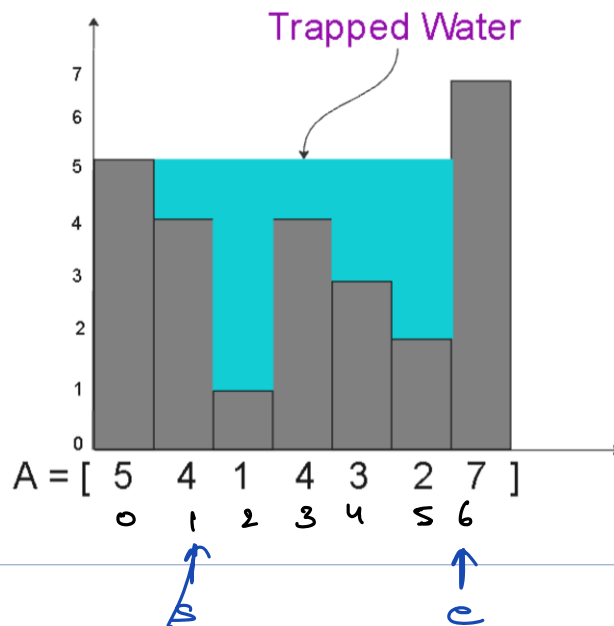
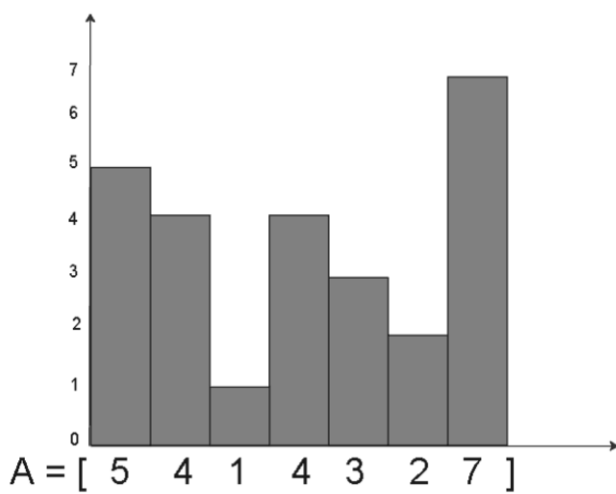
totalWater = totalWater + waterAti

return totalWater



TC: $O(N^2)$

SC: $O(1)$



$h[s]$	$h[e]$	lm	rm	water
5	7	5	0	$0 + 5 - 5 = 0$
4	7	5	0	$0 + 5 - 4 = 1$
1	7	5	0	$1 + 5 - 1 = 5$
4	7	5	0	$5 + 5 - 4 = 6$
3	7	5	0	$6 + 5 - 3 = 8$
2	7	5	0	$8 + 5 - 2 = 11$
7	7	→ stop		

$s < e$
 if $h[s] \leq h[e]$
 if $h[s] \geq lm$
 $lm = h[s]$

else
 $w += lm - h[s]$

$s++$

else
 if $h[e] \geq rm$
 $rm = h[e]$

else:
 $water += rm - h[e]$

$e--$

Optimized

function trapWater(height):

start = 0 // left pointer

end = length of height - 1 // right pointer

leftMax = 0 // maximum height seen from left

rightMax = 0 // maximum height seen from right

totalWater = 0

// Continue until both pointers meet

while start < end:

// If left bar is smaller, water depends on leftMax

if height[start] <= height[end]:

if height[start] >= leftMax:

leftMax = height[start] // update left boundary

else:

totalWater += leftMax - height[start] // trap water

start = start + 1 // move left pointer

// If right bar is smaller, water depends on rightMax

else:

if height[end] >= rightMax:

rightMax = height[end] // update right boundary

else:

totalWater += rightMax - height[end] // trap water

end = end - 1 // move right pointer

return totalWater

